

Ca-Ching! — Understanding Cache Memory

by Barreo, Carlo Gabriel; Gregorio, Gaibril Kyle; Eleydo, Renzel Vince; Leano, Jeremy; Rebudiao, Daniel Christian · 10 minute read

Have you ever noticed that your computer feels fast when running a program, but slows down when doing something new? That speed difference is largely thanks to a small but incredibly clever piece of hardware called **cache memory**. This exhibit explains what it is, how it works, and why it matters — no computer science degree required.

Part 1: Why Does Cache Memory Exist?

The Speed Problem

Modern computer processors are extraordinarily fast. But the main memory (RAM) where data is stored is comparatively slow. When a processor runs at 2.0 GHz, it can do work every **0.5 nanoseconds**. RAM, however, takes **50-70 nanoseconds** to respond. That's like a chef who can cook a dish in 1 second, but has to wait 2 minutes every time they need an ingredient from the pantry.

If the processor had to wait for RAM every single time, it would sit idle for **100 clock cycles per request** — doing nothing but waiting. This bottleneck is called the **Memory Wall**.

Cache memory is the solution. It is a small, ultra-fast memory built directly onto the processor chip. It stores copies of the data that the processor uses most often, so it can be retrieved in **0.5-2.5 nanoseconds** instead of 50-70.

Memory Type	Where It's Used	Speed	Cost
Cache (SRAM)	Inside the processor chip	0.5 – 2.5 ns ⚡	Very expensive
Main Memory (RAM)	Sticks on the motherboard	~ 70 ns	Affordable
SSD Storage	Drive in your computer	5,000 – 50,000 ns	Cheap
Hard Drive (HDD)	Spinning disk storage	> 5,000,000 ns 🐢	Very cheap

How Does the Cache Know What to Store?

The cache is tiny — it can only hold a fraction of what's in RAM. So it has to be smart about what it keeps. It relies on two powerful observations about how software actually behaves:



Analogy: The Researcher's Desk

Picture a researcher in a huge library. The library shelves = **main memory (RAM)**. The researcher's small desk = **cache**. The researcher = **the CPU**.

Every time they need a fact, they walk to the shelves, find the book, and bring it back. That walk takes time. But once a book is on the desk, finding anything in it is instant.

Since the desk is small, they can't bring every book. So they make smart predictions about which books they'll need next. Those predictions follow two patterns:

- Temporal Locality:** "I just used this book — I'll probably need it again soon." (*The same data tends to be re-used in loops.*)
- Spatial Locality:** "I'm reading about Physics — I'll probably need the Chemistry book next to it too." (*Nearby memory addresses tend to be needed together.*)

Because of these patterns, when the cache fetches one piece of data, it also grabs its neighbors — loading a whole "block" at once.

Cache Hits and Misses

Every time the processor needs data, one of two things happens:

- ✅ **Cache Hit** — The data is already in cache. The processor gets it instantly (~2 ns).
- ❌ **Cache Miss** — The data isn't in cache. The processor must wait for RAM (~100 ns). This wait is called the **miss penalty**.

Why does this matter numerically? Consider a program with 130 memory operations:

- Without cache:** Every access hits RAM → **1,300 clock cycles** of waiting
- With cache** (95% hit rate): Most accesses are instant → only **778 clock cycles**

The cache **cuts wait time nearly in half** just by intercepting most requests before they reach RAM.

How Cache Handles Writing Data

Reading from cache is straightforward. But what about *writing*? When the processor changes a value, the cache has a copy — but RAM doesn't know yet. Two strategies handle this:

Write-Through

Also called: Store-Through

Every write to cache is **immediately also written to RAM**. Cache and RAM are always in sync.

✓ **Safe** — no data lost on a crash or power failure.

✗ **Slow** — every single write must wait for RAM (adds ~10 cycles each time).

Write-Back

Also called: Write-Behind

Writes go to cache only. The processor keeps working. RAM is updated **later**, when that cache slot is needed for new data. Modified blocks are flagged with a **"dirty bit"**.

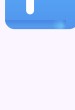
✓ **Fast** — the processor never waits for RAM on writes.

✗ **Risk** — if power is lost before the dirty block is saved, data is gone.

Part 2: How Cache Decides Where to Store Data

Cache is much smaller than RAM, so many RAM locations have to share the same cache slots. The system uses **mapping functions** to decide which slot each piece of data can go into. Think of it like assigning office desks to a large staff.

Direct Mapping — Every Person Has One Assigned Desk



Analogy: The Assigned Parking Lot

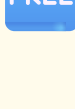
Imagine a parking lot with **4 spaces** and **16 employees**. Each employee is permanently assigned to a space using simple math: *Employee number ÷ 4, take the remainder*.

Employee 0 → Space 0. Employee 4 → Space 0 (wraps around). Employee 13 → Space 1.

Finding a car is lightning fast — you always know exactly which space to check. But there's a big problem: Employees 0, 4, 8, and 12 all share Space 0. If your program needs data from two of those at the same time, they'll keep kicking each other out. This constant eviction is called **thrashing** — and it can make the cache almost useless.

In technical terms: The cache slot number = (RAM block number) mod (number of cache slots). It's fast and simple, but vulnerable to thrashing when frequently-used data happens to map to the same slot.

Fully Associative — Anyone Can Park Anywhere

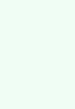


Analogy: Open Parking — No Reserved Signs

All the assigned signs are removed. Everyone just parks in the first open space they find. **No more conflicts** — employees 0 and 4 can both be parked at the same time.

The catch: When you need to find a specific car, you can't go straight to one spot. You have to check *every single space* simultaneously. This requires expensive parallel hardware ("comparators") for every slot. As cache size grows, this becomes prohibitively costly — so it's only used for tiny, specialized caches like Translation Lookaside Buffers (TLBs).

Set-Associative — VIP Zones with Open Spots Inside



Analogy: Parking Zones — The Best of Both Worlds

The lot is divided into **Zones** (called Sets). Each zone has a few open spots. An employee is assigned to a Zone (using the same simple math as before), but within that zone they can take **any available spot**.

Employee 1 → Zone 1. Employee 5 → Zone 1. Both can park at the same time — no conflict. And when you need to find someone, you only check spots *within their zone*, not the whole lot. Fast search, no thrashing.

This is what modern CPUs actually use — typically **4-way, 8-way, or 16-way** set-associative caches.

Mapping Type	Where Can Data Go?	Search Speed	Risk of Thrashing
Direct Mapping	One specific slot only	Very Fast	High — thrashing common
Fully Associative	Any available slot	Slow (check everything)	None — perfect flexibility
Set-Associative ⭐	Any slot within a set	Fast (check one set)	Low — the modern standard

Part 3: When Cache is Full — Choosing What to Remove

When the cache is completely full and new data needs to come in, something already in the cache has to be removed to make room. This evicted block is called the **"victim"**. Which block gets removed is decided by a **replacement algorithm**.

Least Recently Used (LRU) — Remove the Most Forgotten

The most common strategy: **evict the block that hasn't been used for the longest time**. The logic: if you haven't touched it in a while, you probably don't need it anymore.

Example: Cache holds 4 blocks. We request: A, B, C, D, E, D, F

- Cache fills up: A B C D
- Request E → miss. A was used longest ago → **A evicted**. Cache: E B C D
- Request D → **hit** (D was recently used, so it stayed)
- Request F → miss. B was used longest ago → **B evicted**. Cache: E F C D

LRU works well for most programs. The downside: tracking usage order for every block requires extra hardware — which gets expensive in very large caches.

Most Recently Used (MRU) — Remove the Freshest

This sounds backwards — why remove what you just used? But it solves one specific problem where LRU completely fails: **loops that are bigger than the cache**.

❌ LRU Fails: 0% Hit Rate on a Loop

Cache holds **4 blocks**. Program loops through **5 blocks**: A→B→C→D→E→A→B→C→D→E...

- Cache fills: **A, B, C, D**
- Request E → miss. LRU evicts A (oldest). Cache: E, B, C, D
- Request A → miss! LRU evicts B. Cache: E, A, C, D
- Request B → miss! LRU evicts C. Cache: E, A, B, D

LRU keeps evicting exactly the block it will need next. Every access is a miss — 0% hit rate. The cache is useless.

✅ MRU Solves It: Same Loop, High Hit Rate

MRU evicts the *most recently used* block instead:

- Cache fills: **A, B, C, D**
- Request E → miss. MRU evicts D (most recent). Cache: A, B, C, E
- Request A → **HIT!** (A stayed)
- Request B → **HIT!**
- Request C → **HIT!**

By sacrificing the newest block, the older "loop body" blocks stay in cache. Hit rate is now high. MRU is ideal for streaming media, large database scans, and circular buffers.

FIFO and Random — Simpler Alternatives

- FIFO (First In, First Out):** Evict whoever has been in cache the longest — like a queue. Simple, but it can blindly remove a heavily-used block just because it arrived early.
- Random Replacement:** Pick a victim at random. Surprisingly effective! For large, highly-associative caches (8-way or 16-way), random replacement achieves nearly the same hit rate as LRU — with a fraction of the hardware cost.

INTERACTIVE EXHIBIT

Try It Yourself — Live Cache Simulator

Enter a hex memory address (like 0x1A4) and watch how the CPU checks the cache. See a **cache hit** (fast!) or a **cache miss** (slow trip to RAM) — and watch the latency counter change in real time.

CPU CACHE VISUALIZER

Ca-Ching! — The Cache Visualizer

Watch the CPU race against the hardware speed gap. Fetch a memory address and see whether it's a lightning-fast cache hit or a costly trip down to main memory.

🔧 Simulator Controls

Enter Hex Memory Address

0x1A4

Auto Force Hit Force Miss

Fetch Data

BINARY CONVERSION

0001 1010 0100

Sliced Bits

Tag 0001101 Index 001

Offset 00

The index picks cache row #1; the tag is compared to confirm the block is the one we want.

📖 Exhibit Notes

The Hardware Speed Gap

A modern CPU can execute billions of instructions per second, but main memory (RAM) is comparatively slow to respond. Reaching out to RAM can cost ~100 ns, while the on-chip L1 cache answers in ~2 ns. Caches exist to bridge this enormous speed gap so the CPU spends less time waiting and more time computing.

Cache Hits vs. Cache Misses

Principle of Locality (Spatial & Temporal)

CURRENT STATE

🟢 HIT

Cache Hit!

LATENCY COUNTER

2 ns

Time the CPU waited for this fetch

CPU Requests data

↓

HIT

L1 Cache

Row #1: 0x0

Row #2: 0x1

Row #3: 0x2

Row #4: 0x3

8 direct mapped rows - tag shown when occupied

Main Memory (RAM)

~100 ns

Part 4: The Researchers Behind the Theory

The concepts in this exhibit are drawn from landmark computer architecture textbooks that are the global standard in universities worldwide.

David A. Patterson (UC Berkeley) and **John L. Hennessy** (Stanford) pioneered RISC processor design and co-authored the foundational texts used in this exhibit. Their work earned them the **Turing Award** — the Nobel Prize of computing — in 2017.

Carl Hamacher, **Zvonko Vranesic**, and **Safwat Zaky** authored the hardware-focused companion textbook that details how cache circuits are physically built and wired.

References

- D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5th ed. Waltham, MA, USA: Morgan Kaufmann, 2014.
- J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.
- C. Hamacher, Z. Vranesic, S. Zaky, and N. Manjikian, *Computer Organization and Embedded Systems*, 6th ed. New York, NY, USA: McGraw-Hill Education, 2012.
- "Cache Memory: Mapping & Replacement Techniques," *Scribd*, accessed Jul. 20, 2026. [Online]. Available: <https://www.scribd.com/presentation/970494051/Lecture-2-3-4-Cache-Memory-Mapping-Functions-Replace-Algorithms-Write-Policies>
- "Lecture 6: The Memory Hierarchy," Ben-Gurion University, accessed Jul. 20, 2026. [Online]. Available: <https://www.eng.bgu.ac.il/emanad/files/2023/06/Lecture-6-Memory-2023.pdf>
- "Evaluation of the Impact of Coherence Protocols and Cache Sizes on Parallel Algorithms," *SSCAD*, accessed Jul. 20, 2026. [Online]. Available: https://sol.sbc.org.br/index.php/sscad_estendido/article/download/30364/307677

CSARCH2 Virtual Exhibit — "Computer Architecture is Forever"